Sabancı University
CS310 – Mobile Application Development
2026-2027 Fall Project Step 1 Proposal

SUnite Find your group at Sabancı

Group 32

Azra At 34090
Ece Ulutaş 35589
Elif Çamlıkaya 35191
Mert Ali İlter 34447

GitHub: `https://github.com/ssinix/sunite`

1 App Overview

What the app does: SUnite is a mobile app, exclusive to Sabancı University students, for finding and forming groups of any kind. It is organised as a set of category boards on which students can post and browse listings of two kinds: *existing groups looking for new members*, and *students looking for a group to join*. Boards cover course groups, Erasmus, startups/projects and campus events.

Problem it addresses: Finding the right people at SU is currently scattered across WhatsApp groups, SUCourse forums and word of mouth. A student who needs a project partner for a course, wants to meet others going to the same Erasmus school, is looking for co-founders with a specific skill, or simply does not want to go to a club event alone has no single place to look. SUnite gathers all of these requests in one place and makes them searchable by course, school, skill and event.

Intended users: Sabancı University undergraduate and graduate students, verified through their `@sabanciuniv.edu` e-mail address. Clubs and student initiatives benefit indirectly, since students can organise groups around their events.

2 Core Features

1. **SU-only authentication:** sign up and log in with a university e-mail address, with e-mail verification.
2. **User profiles:** name, e-mail, year of study, major(s), minor(s), skills/specialties and current groups; users can view each other's profiles.
3. **Four category boards:** *Course Groups*, *Erasmus*, *Startups & Projects* and *Events*, all built on one shared post/group structure with a category field.
4. **Two post types:** "group looking for members" and "person looking for a group".
5. **Create, edit and delete posts/groups:** title, description, capacity (current/maximum), "looking for" (specific qualities or skills needed) and a category-specific field (course code, Erasmus school and term, or event name and date).
6. **Browse, search and filter:** filter a board by course code, school, tag or groups that still have open spots.
7. **Join requests:** a student requests to join a group; the group owner accepts or declines. Capacity updates automatically and the post closes when the group is full.
8. **My Groups page:** groups the user owns or belongs to, their members and pending join requests.

Optional Features (if time permits)

- **Random matchmaking:** "match me" with other students who are also looking for a group for the same course or event.
- **Friends:** add friends and see which groups they are in.
- **In-app group chat** for members of a group.
- **Push notifications** for new join requests and acceptances.
- **Course list autocomplete** from the SU course catalogue.

3 Data Requirements

The app requires a cloud database so that posts and groups are shared between all users in real time. We plan to use **Firebase Authentication** for accounts and **Cloud Firestore** (NoSQL) for application data. No external API is required for the core features. Small local preferences (e.g. last used filters) will be kept on the device with `shared_preferences`.

Collection	Main fields
`users`	uid, name, e-mail, year, majors, minors, skills, group IDs
`groups`	title, description, category, categoryDetail, capacity (current/max), owner ID, member IDs, "looking for"
`posts`	type (group seeking members / person seeking group), category, categoryDetail, author ID, linked group ID (empty for "person seeking group"), text, created at, status (open/closed)
`joinRequests`	group ID, user ID, status (pending/accepted/declined), created at

Optional: `friendships` (user pairs), `messages` (group chat).

How posts and groups relate. A *group* is the actual team: it stores the owner, member list and capacity, and is the only place where capacity is kept. A *post* is the listing shown on a board. A "group looking for members" post points to its group through `linked group ID`; a "person looking for a group" post has no linked group. The open/closed status is stored in the post and is set to closed automatically when the linked group reaches its maximum capacity.

All four boards share this single `posts`/`groups` structure. They are told apart by the `category` field (course, Erasmus, startup, event) and one generic `categoryDetail` field (course code, school and term, or event name and date). This lets the same board, create-post and group screens be reused for every category, which keeps the scope manageable.

4 Potential Challenges

- **Restricting access to SU students:** enforcing the e-mail domain and verification, and writing Firestore security rules so users can only edit their own data.
- **Data modelling and queries:** Firestore has limits on multi-field filtering, so board filters will need composite indexes and some data duplication.
- **Consistency:** two students may be accepted into the last spot at the same time; membership and capacity updates must use transactions.
- **State management:** keeping boards, group pages and profiles in sync with real-time updates across many screens.
- **Privacy and moderation:** letting users control which profile details are visible and handling inappropriate posts.

5 Team Responsibilities

Step 1 coordinator: Mert Ali İlter

Member	Assigned tasks
Azra At	Created the repository and wrote Introduction
Ece Ulutas	Wrote Core Features
Elif Çamlıkaya	Wrote Data Requirements
Mert Ali İlter (Coordinator)	Arranged Meetings and Potential Challenges